

Lab Exercise 20 - Deploying a Web Application using AWS ECR and ECS

Shelly Singh Rajput
500125383

B2 DevOps

Objective

Learn how to containerize a web application, push the Docker image to Amazon Elastic Container Registry (ECR), and deploy it as a highly available service using Amazon Elastic Container Service (ECS) with AWS Fargate.

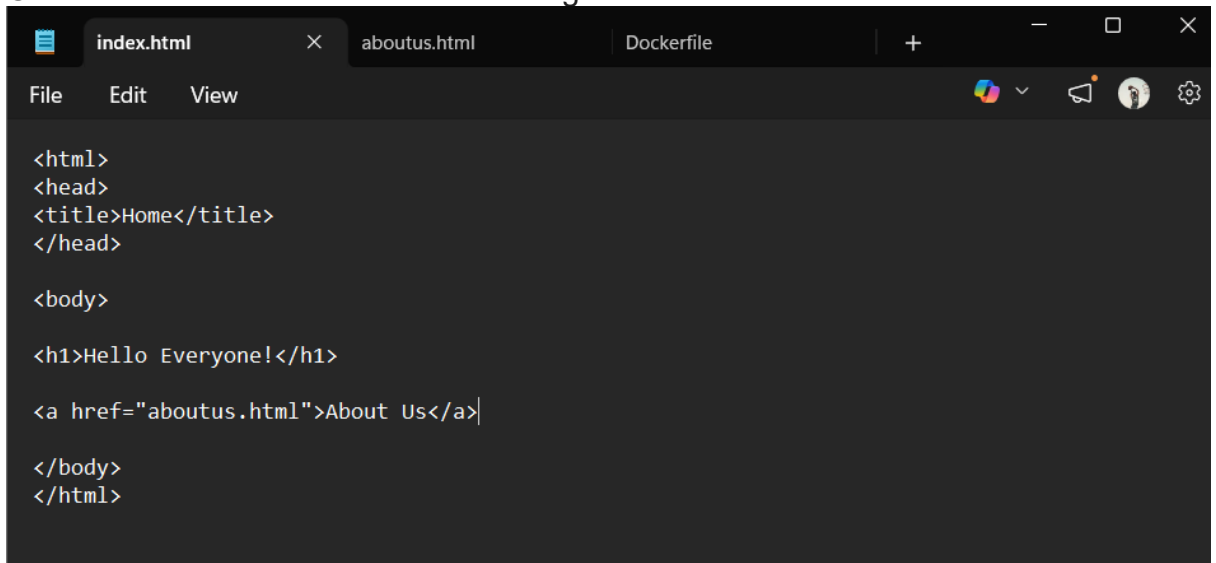
Prerequisites

- AWS Account with administrative privileges.
- AWS CLI installed and configured on your local machine.
- Docker installed and running.
- Basic understanding of Dockerfiles and HTML.

Step 1: Create the Web Application Files

First, prepare the source code and Dockerfile for your web application.

1. Create a directory for your project (e.g., my-web-app) and navigate into it.
2. Create an index.html file with the following content:

A screenshot of a code editor window with a dark theme. The editor has three tabs at the top: 'index.html' (active), 'aboutus.html', and 'Dockerfile'. The 'index.html' tab is selected, and its content is displayed in the main editing area. The content is HTML code for a simple web page. The editor includes a menu bar with 'File', 'Edit', and 'View' options, and a toolbar with various icons for file operations and settings. The code is as follows:

```
<html>
<head>
<title>Home</title>
</head>

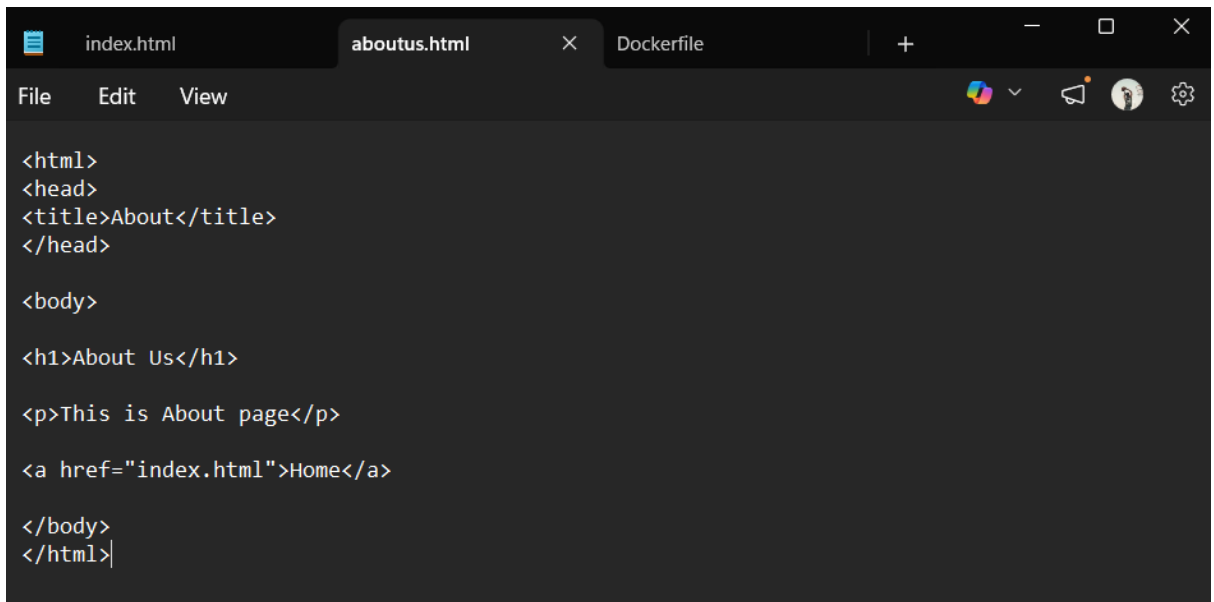
<body>

<h1>Hello Everyone!</h1>

<a href="aboutus.html">About Us</a>

</body>
</html>
```

3. Create an aboutus.html file with the following content:

A screenshot of a code editor with three tabs: index.html, aboutus.html (active), and Dockerfile. The editor shows the HTML content of aboutus.html, which includes a title 'About', a heading 'About Us', a paragraph 'This is About page', and a link to 'index.html' labeled 'Home'.

```
<html>
<head>
<title>About</title>
</head>

<body>

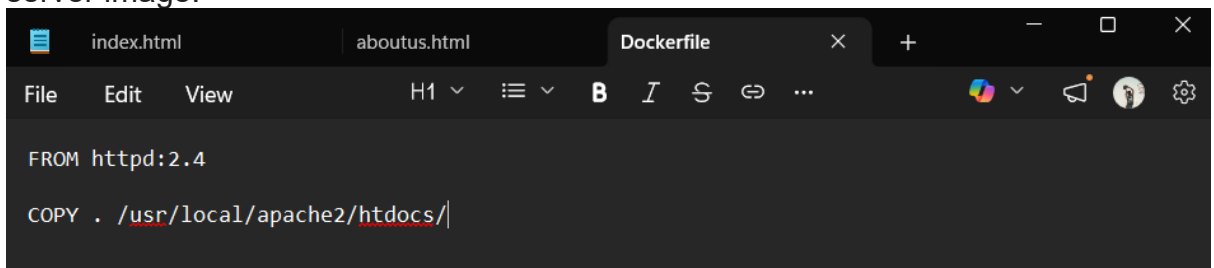
<h1>About Us</h1>

<p>This is About page</p>

<a href="index.html">Home</a>

</body>
</html>
```

4. Create a Dockerfile to serve the HTML files using the official Apache HTTP server image:

A screenshot of a code editor with three tabs: index.html, aboutus.html, and Dockerfile (active). The editor shows the content of the Dockerfile, which consists of two lines: 'FROM httpd:2.4' and 'COPY . /usr/local/apache2/htdocs/'.

```
FROM httpd:2.4

COPY . /usr/local/apache2/htdocs/
```

Step 2: Create an ECR Repository

Create a secure place in AWS to store your Docker images.

1. Log in to the AWS Management Console and navigate to **Amazon Elastic Container Registry (ECR)**.
2. Click **Create repository**.
3. Set the repository visibility to **Private**.
4. Enter lab20 as the **Repository name**.
5. Leave the remaining settings as default (e.g., Mutable tags, AES-256 encryption) and click **Create repository**.
6. Once created, note down the **URI** of your new repository (e.g., 578673727059.dkr.ecr.ap-south-1.amazonaws.com/lab20).

Containers

Amazon Elastic Container Registry

Share and deploy container software, publicly or privately

Amazon Elastic Container Registry (ECR) is a fully managed container registry that makes it easy to store, manage, share and deploy your container images and artifacts anywhere.

Create a repository

Create

Pricing (US)

You only pay for the amount of data you store in your public or private repositories and data transferred to the Internet.

[ECR pricing](#)

Enable private connection to ECR private repositories using VPC interface endpoints and reduce costs.

[Find out more](#)

[Deploy VPC interface endpoints](#)

How it works



Benefits

Fully managed

Secure

Getting started

What is Amazon ECR?

Getting started with ECR

Set up a CI/CD pipeline with ECR

Amazon ECR > Private registry > Repositories > Create a private repository

General settings

Repository name

Enter a concise name. Repositories support namespaces, which you can use to group similar repositories.

578673727059.dkr.ecr.ap-south-1.amazonaws.com/lab20

5 out of 256 characters maximum (2 minimum). The name must start with a letter and can only contain lowercase letters, numbers and special characters _./.

Image tag settings

Image tag mutability

Choose the tag mutability setting.

☒ Mutable
Image tags can be overwritten

☐ Immutable
Image tags can't be overwritten.

Mutable tag exclusions

Tags that match these filters will be immutable (can't be overwritten). Using wildcards (*) will match zero or more image tag characters.

Add filter

Filters must only contain letters, numbers, and special characters (_.*). Each filter is limited to 128 characters, 2 wildcards (*), and you can add up to 5 filters in the exclusion list.

Encryption settings

The encryption settings for a repository can't be changed once the repository is created.

Encryption configuration

By default, repositories use the industry standard Advanced Encryption Standard (AES) encryption. You can optionally choose to use a key stored in the AWS Key Management Service (KMS) to encrypt the images in your repository.

☒ AES-256
Industry standard Advanced Encryption Standard (AES) encryption

☐ AWS KMS
AWS Key Management Service (KMS)

Amazon ECR > Private registry > Repositories

Amazon Elastic Container Service

Private registry

Repositories

Features & Settings

Public registry

Repositories

Settings

ECR public gallery

Amazon ECS

Amazon EKS

Getting started

Documentation

Successfully created private repository, lab20

Private repositories (1)

View push commands

Delete

Actions

Create repository

Search by repository substring

Repository name	URI	Created at	Tag mutability
lab20	071179620453.dkr.ecr.eu-north-1.amazonaws.com/lab20	April 17, 2026, 01:36:44 (UTC+05.5)	Mu

Step 3: Build and Push the Docker Image

Use your local terminal to build the image and push it to the ECR repository created in Step 2.

1. Authenticate your local Docker client to your ECR registry. Replace the URI with your specific account details:

```
aws ecr get-login-password --region ap-south-1 | docker login --username AWS --password-stdin 578673727059.dkr.ecr.ap-south-1.amazonaws.com
```

Expected Output: Login Succeeded

2. Build your Docker image locally:

```
docker build -t lab20 .
```

3. Tag your local image to match your ECR repository URI:

```
docker tag lab20:latest 578673727059.dkr.ecr.ap-south-1.amazonaws.com/lab20:latest
```

4. Push the tagged image to AWS ECR:

```
docker push 578673727059.dkr.ecr.ap-south-1.amazonaws.com/lab20:latest
```

```
C:\Users\ASUS\my-web-app>docker build -t lab20 .
[+] Building 13.6s (8/8) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile              0.1s
=> => transferring dockerfile: 88B                               0.0s
=> [internal] load metadata for docker.io/library/httpd:2.4     4.4s
=> [auth] library/httpd:pull token for registry-1.docker.io     0.0s
=> [internal] load .dockerignore                                 0.1s
=> => transferring context: 2B                                    0.0s
=> [internal] load build context                                0.1s
=> => transferring context: 471B                                  0.0s
=> [1/2] FROM docker.io/library/httpd:2.4@sha256:bdba5c86022f2d6ad07 8.2s
=> => resolve docker.io/library/httpd:2.4@sha256:bdba5c86022f2d6ad07 0.1s
=> => sha256:a838a4be55ced5f2d7208c56a3a13ecbd151105 2.00MB / 2.00MB 1.0s
=> => sha256:4e51448030490b8604d7662b05848e89e3563fe3b95 289B / 289B 1.1s
=> => sha256:8261ede62f8c1c85c3e4b2ae3a9ae5a586b84fcc61b 146B / 146B 1.4s
=> => sha256:61e1798e23ba12f14505e225391338db290ee 13.45MB / 13.45MB 2.6s
=> => sha256:5435b2dcdf5cb7faa0d5b1d4d54be2c72a776 29.78MB / 29.78MB 2.8s
=> => extracting sha256:5435b2dcdf5cb7faa0d5b1d4d54be2c72a776fab9a60 2.6s
=> => extracting sha256:8261ede62f8c1c85c3e4b2ae3a9ae5a586b84fcc61b6 0.0s
=> => extracting sha256:4f4fb700ef54461cfa02571ae0db9a0dc1e0cdb55774 0.0s
=> => extracting sha256:a838a4be55ced5f2d7208c56a3a13ecbd15110574ee3 0.5s
=> => extracting sha256:61e1798e23ba12f14505e225391338db290ee6cecfb6 0.8s
=> => extracting sha256:4e51448030490b8604d7662b05848e89e3563fe3b955 0.0s
=> [2/2] COPY . /usr/local/apache2/htdocs/                     0.6s
=> exporting to image                                           0.6s
=> => exporting layers                                           0.2s
=> => exporting manifest sha256:7596d625662208c46cf99636d5d7f78336ba 0.0s
=> => exporting config sha256:e0bf7fb5087aab0e41a4d1da7156ff61f52f7f 0.0s
=> => exporting attestation manifest sha256:8c8b43c4acafea5d73bd7d8a 0.1s
=> => exporting manifest list sha256:bc49c487059a3770ceb1364dc2ea7fc 0.0s
=> => naming to docker.io/library/lab20:latest                  0.0s
=> => unpacking to docker.io/library/lab20:latest               0.1s
```

```
C:\Users\ASUS\my-web-app>
```

```
C:\Users\ASUS\my-web-app>aws ecr get-login-password --region eu-north-1 | do
cker login --username AWS --password-stdin 071179620453.dkr.ecr.eu-north-1.a
mazonaws.com
Login Succeeded
```

```
C:\Users\ASUS\my-web-app>docker tag lab20:latest 071179620453.dkr.ecr.eu-nor
th-1.amazonaws.com/lab20:latest
```

```
C:\Users\ASUS\my-web-app>docker push 071179620453.dkr.ecr.eu-north-1.amazona
ws.com/lab20:latest
```

```
The push refers to repository [071179620453.dkr.ecr.eu-north-1.amazonaws.com
/lab20]
```

```
5435b2dcdf5c: Pushed
```

```
8261ede62f8c: Pushed
```

```
a838a4be55ce: Pushed
```

```
4e5144803049: Pushed
```

```
740c98f8f440: Pushed
```

```
8d13eb37956e: Pushed
```

```
4f4fb700ef54: Pushed
```

```
61e1798e23ba: Pushed
```

```
latest: digest: sha256:bc49c487059a3770ceb1364dc2ea7fc8dc383002e39078dc3282b
820163eebe5 size: 856
```

```
C:\Users\ASUS\my-web-app>
```

Amazon ECR > Private registry > Repositories > lab20

Amazon Elastic Container Service

Private registry

- Repositories
 - lab20
- Features & Settings

Public registry

- Repositories
- Settings

ECR public gallery [↗](#)

Amazon ECS [↗](#)

Amazon EKS [↗](#)

Getting started [↗](#)

Documentation [↗](#)

lab20

Summary | **Images** | Lifecycle policy | Permissions | Repository tags

Images (3) [Info](#) [Delete](#) [Copy URI](#) [Details](#) [Scan](#) [View push commands](#)

☐	Image tags	Type	Created at	Image size (MB)	Image digest	Last pulled at
☐	latest	Image Index	April 17, 2026, 01:41:13 (UTC+05.5)	45.23	sha256...	-
☐	-	Image	April 17, 2026, 01:41:13 (UTC+05.5)	45.23	sha256...	-
☐	-	Image	April 17, 2026, 01:41:13 (UTC+05.5)	0.00	sha256...	-

Step 4: Create an ECS Cluster

Set up the infrastructure to run your containers.

1. Navigate to **Amazon Elastic Container Service (ECS)** in the AWS Console.
2. Select **Clusters** from the left-hand menu and click **Create cluster**.
3. Set the **Cluster name** to lab20.
4. Under Infrastructure, select **AWS Fargate (Serverless)** to run containers without managing the underlying EC2 instances.
5. Click **Create**.

Amazon Elastic Container Service

Express Mode

Clusters

Namespaces

Task definitions

Daemon task definitions New

Account settings

AWS Batch [i](#)

Amazon ECR [i](#)

Repositories [i](#)

Online learning workshop [i](#)

Documentation [i](#)

Discover products [i](#)

Subscriptions [i](#)

Tell us what you think

Containers

Amazon Elastic Container Service

Fully managed containers

Amazon Elastic Container Service (Amazon ECS) is a highly scalable and fast container management service that makes it easy to run, stop and manage containers on a cluster.

Deploy your containerised applications

Amazon ECS makes it easy to deploy, manage and scale Docker containers running applications, services and batch processes.

Get started

Pricing (US)

Amazon ECS on AWS Fargate [Fargate pricing](#)

Amazon ECS Managed Instances [ECS Managed Instances pricing](#)

Amazon ECS on Amazon EC2 [EC2 pricing](#)

Amazon ECS Anywhere [ECS Anywhere pricing](#)

More resources

[Documentation](#)

How it works

Introduction to Amazon Elastic Container Service

Amazon Web Services

Watch on YouTube

Amazon Elastic Container Service

Express Mode

Clusters

Namespaces

Task definitions

Daemon task definitions New

Account settings

AWS Batch [i](#)

Amazon ECR [i](#)

Repositories [i](#)

Online learning workshop [i](#)

Documentation [i](#)

Discover products [i](#)

Subscriptions [i](#)

Tell us what you think

Clusters

Introducing managed daemons

You can now manage software agents as standalone daemons across container instances in a capacity provider, independent of your applications. [Find out more](#)

Create daemon task definition

Clusters (0) [Info](#)

Search clusters

By default, we only load up to 1,000 clusters at a time.

Cluster Services Tasks Container instances CloudWatch monitoring Capacity provider strategy

No clusters

Create cluster

Last updated 15 April 2026, 23:34 (UTC+5:30)

Create cluster

Amazon Elastic Container Service

Express Mode

Clusters

Namespaces

Task definitions

Daemon task definitions New

Account settings

AWS Batch [i](#)

Amazon ECR [i](#)

Repositories [i](#)

Online learning workshop [i](#)

Documentation [i](#)

Discover products [i](#)

Subscriptions [i](#)

Tell us what you think

Create cluster

Create cluster [Info](#)

An Amazon ECS cluster groups together tasks and services, and allows for shared capacity and common configurations. All of your tasks, services and capacity must belong to a cluster.

Cluster configuration

Cluster name

lab20

Cluster name must be 1 to 255 characters. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).

Service Connect defaults - optional

Infrastructure - advanced [Info](#)

Configure the manner of obtaining compute resources that will be used to host your application.

Select a method of obtaining compute capacity

Your cluster is automatically configured for AWS Fargate (serverless), but you may choose to add Amazon EC2 instances (servers).

Fargate only

Serverless - you don't think about creating or managing servers. Great for most common workloads.

Fargate and managed instances

Managed instances - Amazon ECS will manage patching and scaling on your behalf while giving you configurability about the types of instances. Great for more advanced workloads.

Fargate and Self-managed instances

Self-managed instances - you must ensure the instances are patched and scaled properly, and you have full control over the instances.

Monitoring - optional

Configure observability, encryption and logging options to maintain compliance and operational visibility of your container environment.

Encryption - optional

Choose the KMS keys used by tasks running in this cluster to encrypt your storage.

✓ Cluster lab20 has been created successfully.



[View cluster](#)

Introducing managed daemons



You can now manage software agents as standalone daemons across container instances in a capacity provider, independent of your applications. [Learn more](#)

[Create daemon task definition](#)

Clusters (1) [Info](#)

Last updated April 17, 2026, 01:50 (UTC+5:30)



[Create cluster](#)

By default, we only load up to 1,000 clusters at a time.

< 1 >

Cluster	Services	Tasks
lab20	0	No tasks running

< 1 >

Step 5: Create a Task Definition

Define how your container should be configured when it runs.

- In ECS, select **Task definitions** from the left menu and click **Create new task definition**.
- Set the **Task definition family** name to lab20-task.
- Under Infrastructure requirements, select **AWS Fargate** as the launch type.
- Set the Operating system/Architecture to **Linux/X86_64**.
- Configure the Task size:
 - CPU:** 1 vCPU
 - Memory:** 3 GB
- Under Container - 1, configure the following:
 - Name:** lab20-container
 - Image URI:** Paste the URI of your ECR image (e.g., 578673727059.dkr.ecr.ap-south-1.amazonaws.com/lab20:latest).

- **Container port: 80**
- **Protocol: TCP**

7. Click **Create**.

Amazon Elastic Container Service > Task definitions

Express Mode
Clusters
Namespaces
Task definitions
Daemon task definitions [New](#)
Account settings

AWS Batch [L2](#)
Amazon ECR [L2](#)
Repositories [L2](#)

Online learning workshop [L2](#)
Documentation [L2](#)
Discover products [L2](#)
Subscriptions [L2](#)

[Tell us what you think](#)

Task definitions (0) [Info](#)

Last updated 15 April 2026, 23:36 (UTC+5:30) [Refresh](#) [Deploy](#) [Create new revision](#) [Create new task definition](#)

Filter status: Active

Task definition | Status of last revision

No task definitions
No task definitions to display.
[Create new task definition](#)

Amazon Elastic Container Service > Create new task definition

Express Mode
Clusters
Namespaces
Task definitions
Daemon task definitions [New](#)
Account settings

AWS Batch [L2](#)
Amazon ECR [L2](#)
Repositories [L2](#)

Online learning workshop [L2](#)
Documentation [L2](#)
Discover products [L2](#)
Subscriptions [L2](#)

[Tell us what you think](#)

Create new task definition [Info](#)

Task definition configuration

Task definition family [Info](#)
Specify a unique task definition family name.
lab20-task
Up to 255 letters (uppercase and lowercase), numbers, hyphens, and underscores are allowed.

Infrastructure requirements [Info](#)
Specify the infrastructure requirements for the task definition.

Launch type [Info](#)
Selection of the launch type will change task definition parameters.

☒ **AWS Fargate**
Serverless compute for containers.

☐ **Managed instances**
Use if you have specific hardware constraints, such as GPU accelerators, CPU instruction sets, or network-optimized hardware (offloads scaling, patching, and instance management to AWS).

☐ **Amazon EC2 instances**
Self-managed infrastructure using Amazon EC2 instances.

OS, Architecture, Network mode
Network mode is used for tasks and is dependent on the compute type selected.

Operating system/Architecture [Info](#): Linux/X86_64

Network mode [Info](#): awsvpc

Task size [Info](#)
Specify the amount of CPU and memory to reserve for your task.

CPU: 1 vCPU

Memory: 3 GB

Amazon Elastic Container Service > Create new task definition

Express Mode
Clusters
Namespaces
Task definitions
Daemon task definitions [New](#)
Account settings

AWS Batch [L2](#)
Amazon ECR [L2](#)
Repositories [L2](#)

Online learning workshop [L2](#)
Documentation [L2](#)
Discover products [L2](#)
Subscriptions [L2](#)

[Tell us what you think](#)

Container - 1 [Info](#)

Container details
Specify a name, container image and whether the container should be marked as essential. Each task definition must have at least one essential container.

Name: lab20-container

Essential container: Yes

Image URI: 578673727059.dkr.ecr.ap-south-1.amazonaws.com/lab20:latest [Browse ECR images](#)
Up to 255 letters (uppercase and lowercase), numbers, hyphens, underscores, colons, periods, forward slashes, and number signs are allowed.

Private registry [Info](#)
Store credentials in Secrets Manager, and then use the credentials to reference images in private registries.

☐ **Private registry authentication**

Port mappings [Info](#)
Add port mappings to allow the container to access ports on the host to send or receive traffic. For port name, a default will be assigned if left blank.

Container port	Protocol	Port name	App protocol
80	TCP	container-port-protocol	HTTP

[Add port mapping](#) [Remove](#)

Read-only root file system [Info](#)
When this parameter is turned on, the container is given read-only access to its root file system.

☐ **Read only**

Resource allocation limits - conditional [Info](#)
Container-level CPU, GPU and memory limits are different from task-level values. They define how many resources are allocated for the container. If the container attempts to exceed the memory specified by the hard limit, the container is terminated.

CPU	GPU	Memory hard limit	Memory soft limit
1 In vCPU	1	3 In GB	1 In GB

✔ **Task definition successfully created**
lab20-task:1 has been successfully created. You can use this task definition to deploy a service or run a task.

[View task definition](#)

Notifications 0 0 2 0 0

lab20-task:1

Last updated
April 17, 2026, 01:53 (UTC+5:30)



[Deploy](#)

[Actions](#)

[Create new revision](#)

Overview [Info](#)

ARN

arn:aws:ecs:eu-north-1:071179620453:task-definition/lab20-task:1

Status

✔ ACTIVE

Time created

April 17, 2026, 01:53 (UTC+5:30)

App environment

Fargate

Task role

-

Task execution role

[ecsTaskExecutionRole](#)

Operating system/Architecture

Linux/X86_64

Network mode

awsvpc

Fault injection

⊖ Turned off

Step 6: Create an ECS Service

Deploy your task definition into the cluster.

1. Navigate back to your lab20 cluster and go to the **Services** tab. Click **Create**.
2. Under Compute configuration, select **Launch type** and ensure **Fargate** is chosen.

- Under Deployment configuration, set the **Task definition family** to lab20-task and select the latest revision.
- Assign a **Service name**, such as lab20-task-service-050smovw.
- Set the desired tasks to 1.
- Click **Create** to deploy the service. Wait a few minutes for the deployment to finish and the task status to turn to **Running**.

Amazon Elastic Container Service > Clusters > lab20 > Create service

Create service info

Amazon Elastic Container Service

Express Mode

Clusters

Namespaces

Task definitions

Daemon task definitions New

Account settings

AWS Batch ↗

Amazon ECR ↗

Repositories ↗

Online learning workshop ↗

Documentation ↗

Discover products ↗

Subscriptions ↗

Tell us what you think

Service details

Task definition family
Select an existing task definition. To create a new task definition, go to [Task definitions](#).

lab20-task

Task definition revision Latest
Select the task definition revision from the 100 most recent entries, or enter a revision. Leave the field blank to use the latest revision.

1

Service name
Assign a service name that is unique for this cluster.

lab20-task-service-050smovw

Up to 255 letters (uppercase and lowercase), numbers, underscores and hyphens are allowed. Service names must be unique within a cluster.

Environment AWS Fargate

Existing cluster

lab20

▼ **Compute configuration - advanced**

Compute options Info
To ensure task distribution across your compute types, use appropriate compute options.

☐ Capacity provider strategy
Specify a launch strategy to distribute your tasks across one or more capacity providers.

☒ Launch type
Launch tasks directly without the use of a capacity provider strategy.

Launch type Info

Amazon Elastic Container Service > ... > Services

lab20-task-service-fs938fft deployment is in progress. It takes a few minutes.

[View in CloudFormation](#)

Notifications 0 0 1 0 1

lab20

Last updated April 17, 2026, 02:02 (UTC+5:30) [Refresh](#) [Actions](#)

[Create with Express Mode](#)

Cluster overview

ARN
 arn:aws:ecs:eu-north-1:071179620453:cluster/lab20

CloudWatch monitoring
 Default

Services

Draining
-

Active
1

Status
 Active

Registered container instances
-

Tasks

Pending
-

Running
-

<

Daemons

New

Tasks

Infrastructure

Metrics

>

Tasks (1)

Last updated
April 17, 2026, 02:21 (UTC+5:30)

Manage tags

Stop ▾

Run new task

Filter tasks by property or value

Filter desired status
Any desired status ▾

Filter launch type
Any launch type ▾

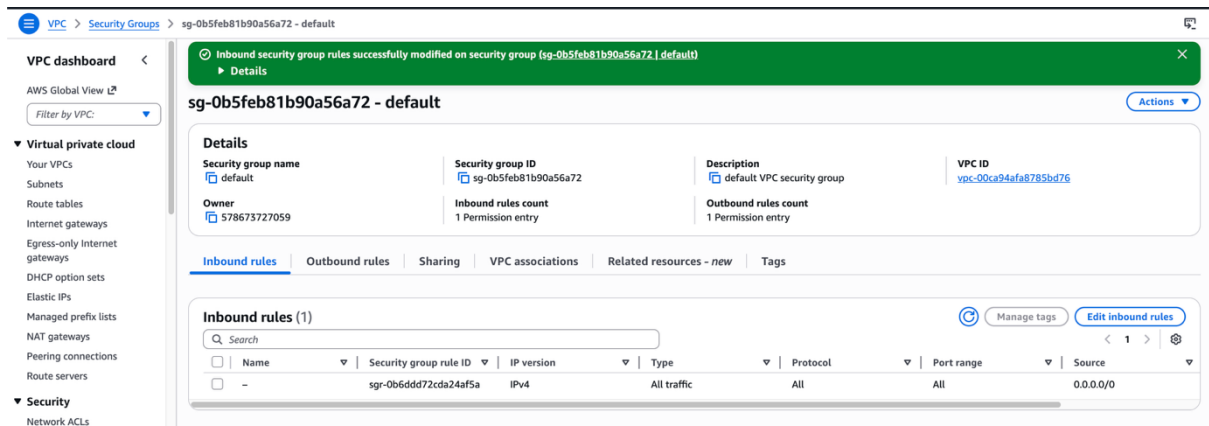
< 1 > ⚙

☐	Task	Last status	Desired sta
☐	 68731e67e688473390b8...	 Running	 Running

Step 7: Configure Security Groups

Ensure public traffic can reach your web server on port 80.

1. Locate the Elastic Network Interface (ENI) or the Security Group associated with your running ECS Task.
2. Open the **VPC Dashboard** or EC2 Security Groups console.
3. Select the relevant security group (e.g., sg-0b5feb81b90a56a72 - default).
4. Edit the **Inbound rules**.
5. Add a rule to allow **All traffic** (or specifically HTTP) on Port **80** from the Source **0.0.0.0/0**.
6. Save the rules.



Step 8: Verify the Deployment

Access your live application over the internet.

1. In the ECS console, navigate to your running task inside the `lab20` cluster.
2. Find the **Public IP** address assigned to the Fargate task
3. Open a new tab in your web browser and navigate to (<http://13.127.18.163>).
4. You should see the "Hello Everyone!" heading. Click the link to verify routing to the "About Us" page.



Hello Everyone!

[About Us](#)



About Us

This is About page

[Home](#)